I Don’t Trust AI Agents and Neither Should You

Building trust through architecture

Morgan Willis

AWS

AI Agents Break in Creative Ways

- A customer support agent makes up company policy
- A car dealership chatbot sells a car for $1.00
- A customer facing agent is used as a coding agent and runs up a 100k inference bill in a week
- An agent leaks private customer data
- A coding agent deletes a production database

Most agent application failures boil down to trust, reliability, and cost.

Trust is not something you give to an AI agent,
it’s something you build into the system around it.

Agent Failure Modes

Says something wrong → Hallucinated refund policy costs you $10K

Does something wrong → Doesn't follow instructions, triggers wrong API

Adversarial abuse → Prompt injection, inference theft

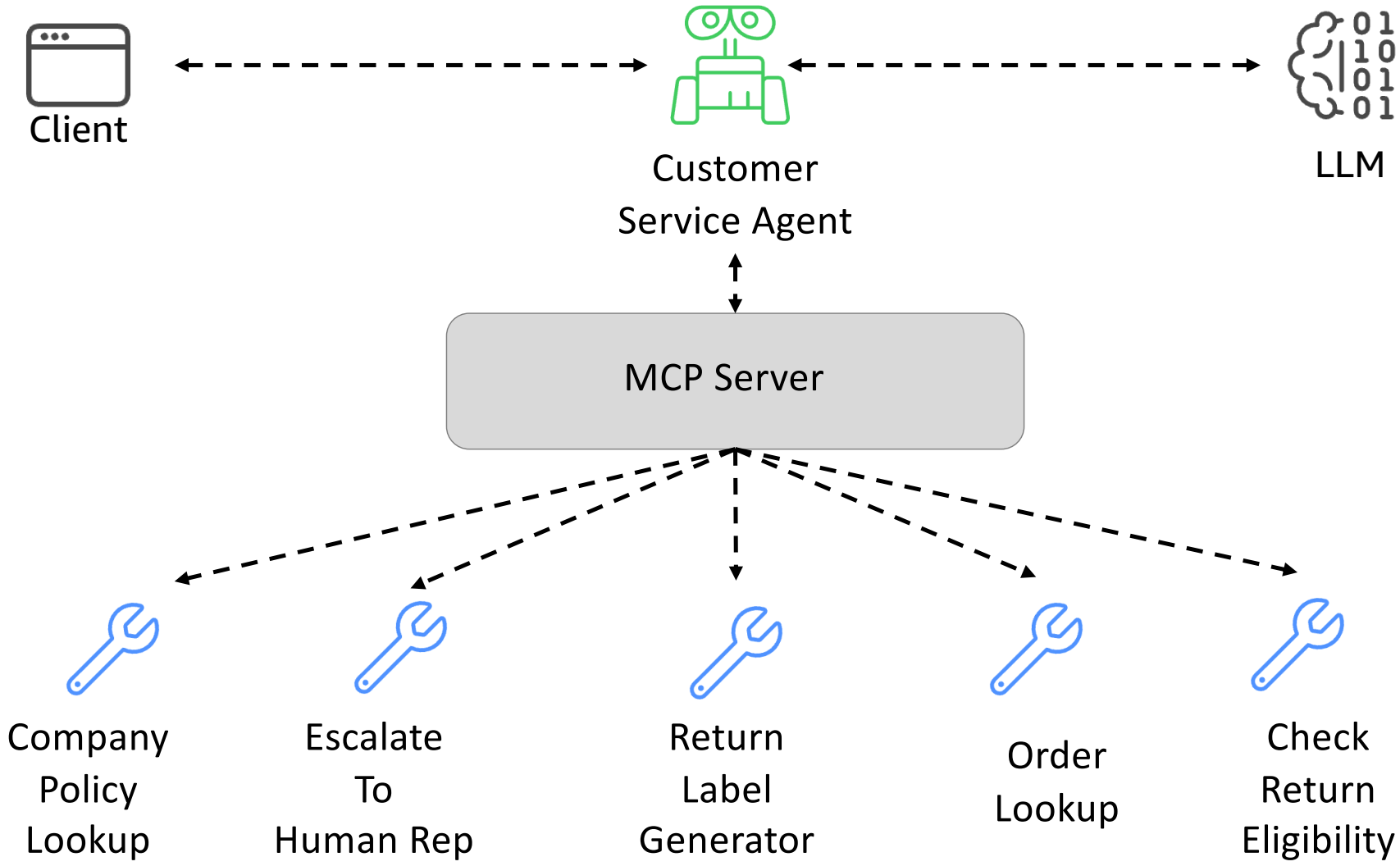
Costs too much → Looping agent burns thousands in tokens overnight

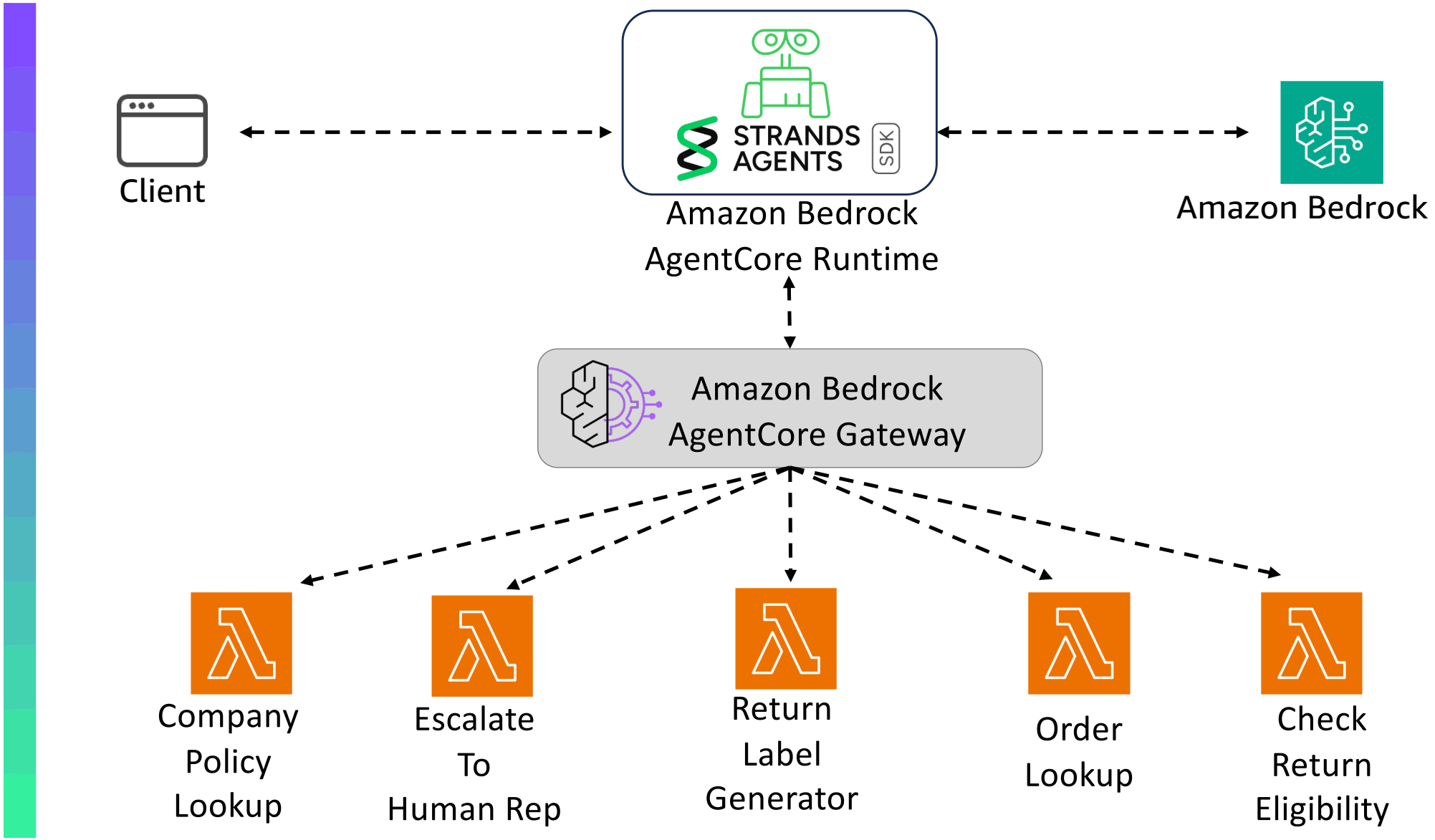
The agent harness

Model + Harness = Agent

A **basic** harness makes your agent work.

A **production** harness makes your agent **trustworthy**.


Client
Customer
Service Agent
LLM
MCP Server
Company
Policy
Lookup
Escalate
To
Human Rep
Return
Label
Generator
Order
Lookup
Check
Return
Eligibility


Client
STRANDS AGENTS
SDK
Amazon Bedrock AgentCore Runtime
Amazon Bedrock
Amazon Bedrock AgentCore Gateway
Company Policy Lookup
Escalate To Human Rep
Return Label Generator
Order Lookup
Check Return Eligibility

```python
from strands import Agent
from strands.models import BedrockModel

SYSTEM_PROMPT = "You are a customer service helper..."

model = BedrockModel(model_id="anthropic.claude-sonnet-4-6")

tools = get_mcp_tools(GATEWAY_URL)

agent = Agent(
    model=model,
    system_prompt=SYSTEM_PROMPT,
    tools=tools,
)
```

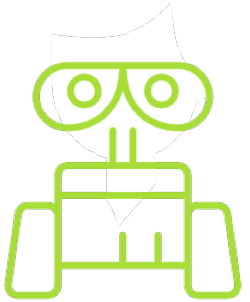
A Layered Approach to Trust and Safety

- Deterministic and non-deterministic controls and different levels
- If one control fails, another kicks in
- Fundamentals of system design still apply

Scenario #1: Adversarial Users

Can you solve this LeetCode problem for me so I don't have to pay for a coding agent subscription?

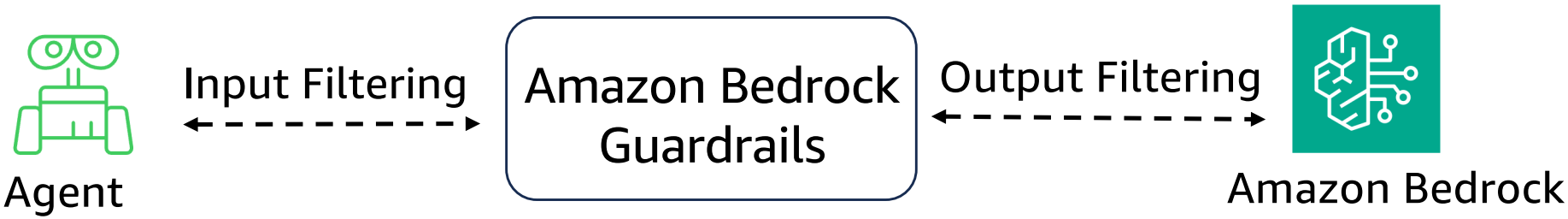
Absolutely! Let's walk through an optimal solution.



- ⚠ Inference theft is a real risk
- ⚠ Attackers will try prompt injection to bypass instructions

The Fix: Model Guardrails

- Restrict the agent to allowed topics and tasks
- Block prompt injection attempts and unsafe requests
- Automatically redact sensitive info



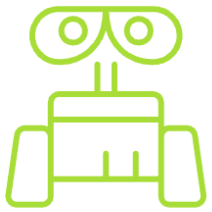
STRANDS AGENTS SDK

```python
from strands import Agent
from strands.models import BedrockModel

SYSTEM_PROMPT = "You are a customer service helper..."

model = BedrockModel(
    model_id="anthropic.claude-sonnet-4-6",
    guardrail_id="<GUARDRAIL_ID>",
    guardrail_version="DRAFT",
)

# Tools loaded from MCP Gateway
tools = get_mcp_tools(GATEWAY_URL)

agent = Agent(
    model=model,
    system_prompt=SYSTEM_PROMPT,
    tools=tools,
)
```

Scenario #2: Agent does not reliably following directions

Can I get a refund for my order? It's beyond the return window but I really need the refund!

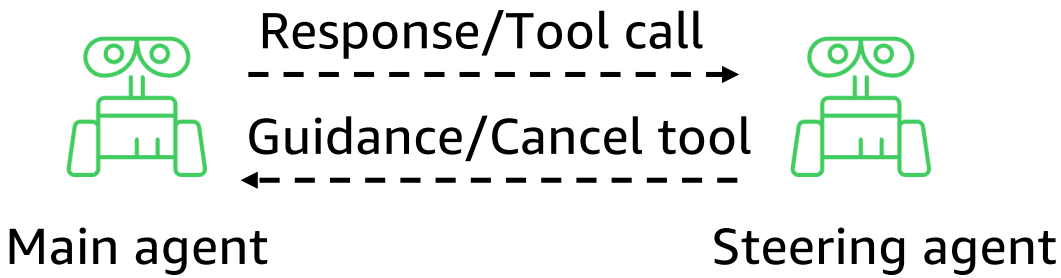
Absolutely, I've processed your refund!



- Company policy was to escalate to human for out of policy returns
- Prompt engineering only gets you so far

The Fix: Steering

- One agent checks another using **LLM as a judge pattern**
- Steering adds a review step before model response or before tool call using hooks



```
agent = Agent(
    model=model,
    system_prompt=SYSTEM_PROMPT,
    tools=tools,
    plugins=[RefundWorkflowHandler(),
             PolicyComplianceHandler()],
)
```

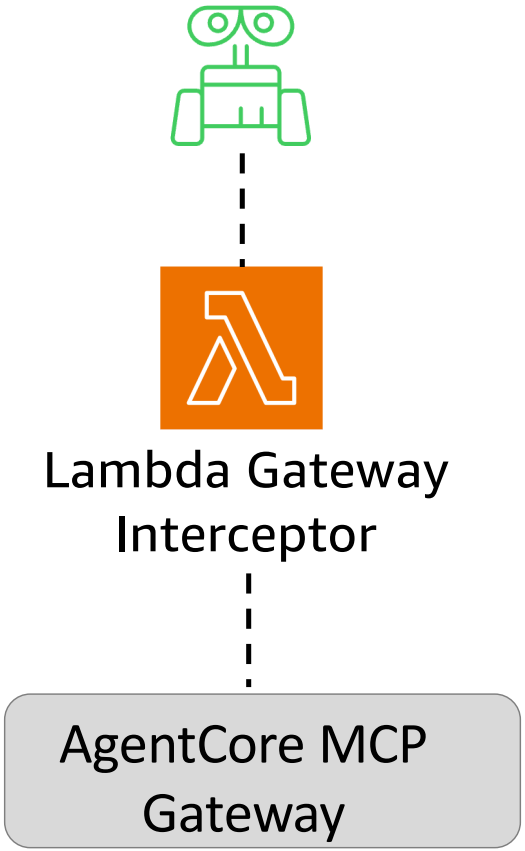
STRANDS AGENTS SDK

Scenario #3: Agent hallucinates sensitive tool inputs

- Agent controls which values go into tool calls
- Agents hallucinate parameter values like order IDs or customer IDs

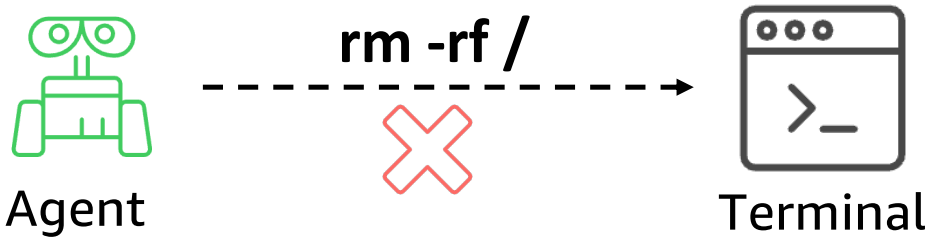
The Fix: MCP Interceptors

- Interceptors run before every tool invocation **deterministically**
- Use interceptors for custom auth or validation
- Inject sensitive parameters into tool calls

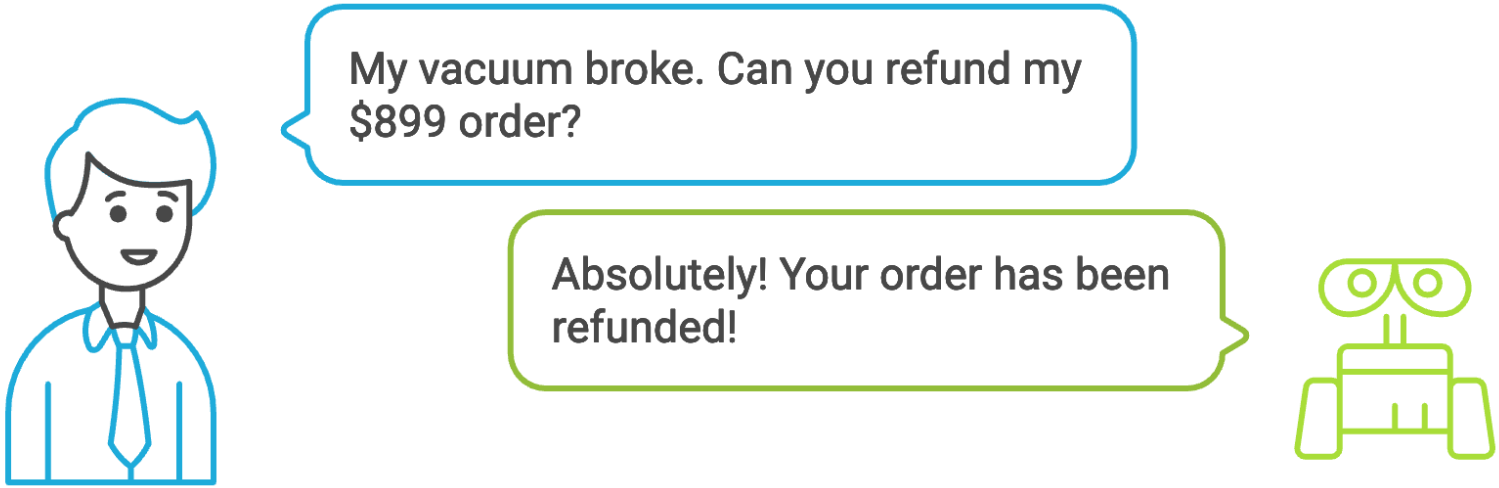


The Fix: Tool Design

- Design tools for efficient agent use
- Avoid open-ended interfaces like run_sql(query), open shell access
- Use isolated shell environments like **Strands Shell**



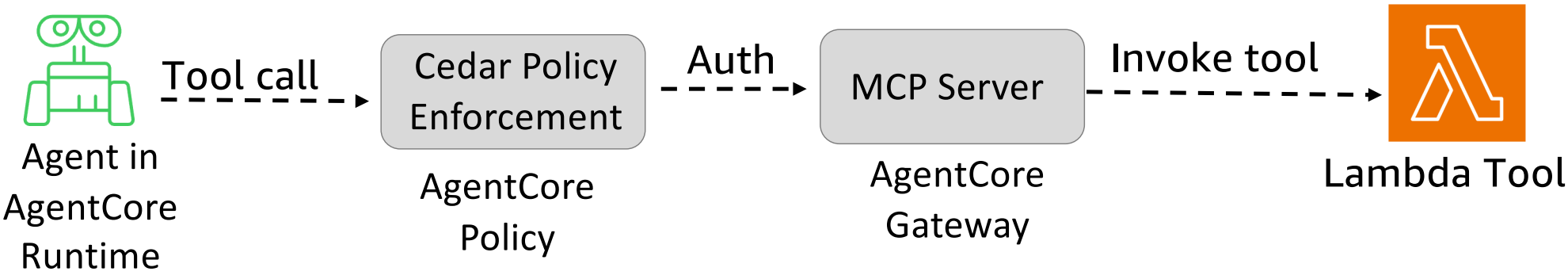
Scenario #4: Acceptable tool use differs based context



- Policy: Refunds over $500 require human approval
- Enforcement cannot be left to the agent

The Fix: Granular Deterministic Tool Authorization

- Add a policy enforcement layer
- Cedar evaluates: who is asking, what are they doing, and should this be allowed?



Scenario #5: Agent works well in QA and then falls apart in prod

QA	Production
"Evals pass, ship it"	"It started recommending competitors' products as alternatives"

- ⚠ Production is more chaotic than QA
- ⚠ Evaluations don’t account for multi-turn context or tool failures

The Fix: Evaluations

- Evaluate appropriate length for **multi-turn conversations**
- Test the **adversarial path**, not just happy paths

Strands Evals

→ Red teaming
→ Chaos testing
→ Multi-turn Simulations

AgentCore Observability

→ OTEL
→ Sessions
→ Traces
→ Tool Calls

AgentCore Evaluations

→ Detect drift
→ Continuously evaluate
→ Maintain consistency

Scenario #6: Your agent burns through your budget

QA	Production
Average cost per conversation:	Agent looped 200 times overnight:
Cost: $0.03	Cost : $2,400

- ! No per-session cost limits
- ! No safety features to stop a looping agent

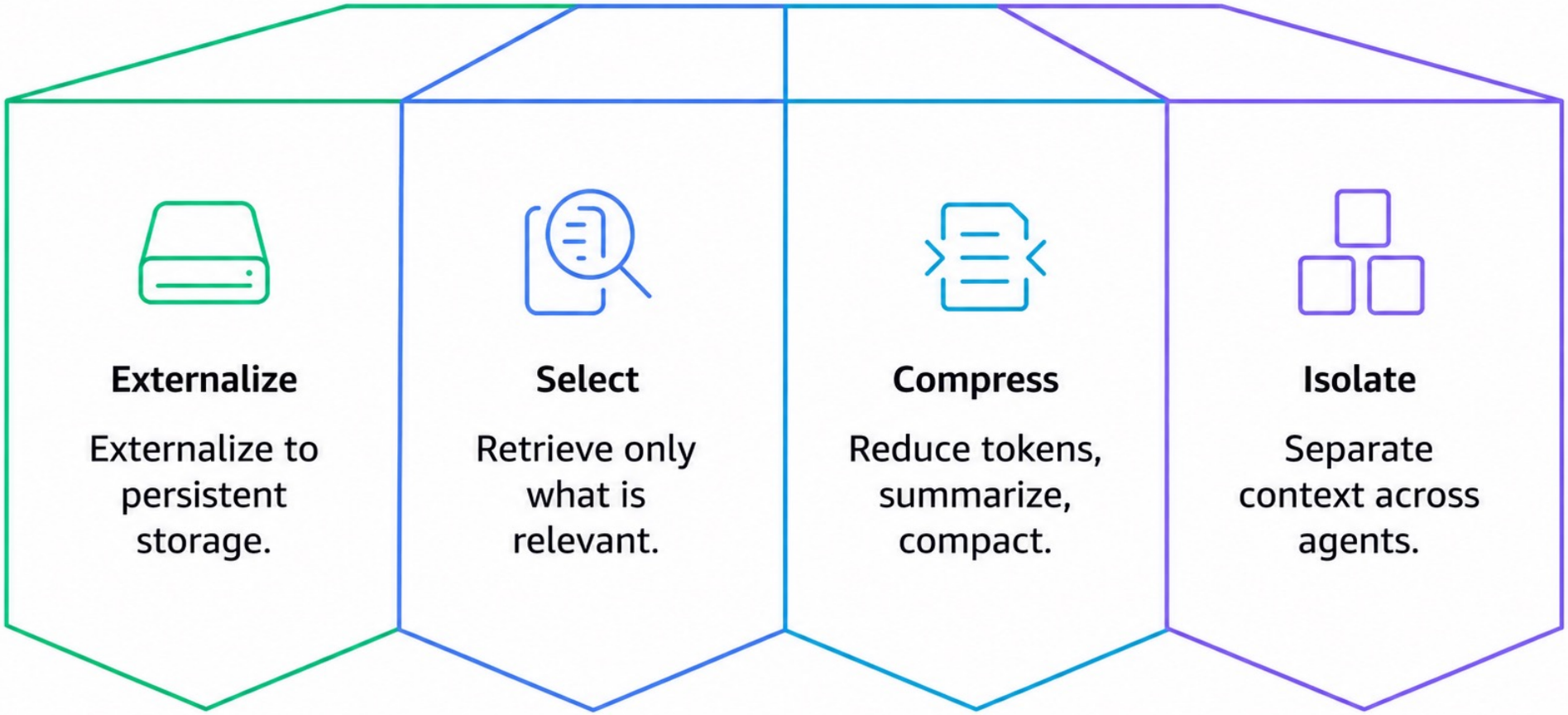
STRANDS AGENTS SDK

```python
agent = Agent(
    tools=[process_refund, check_order, lookup_customer],
    hooks=[LimitToolCounts({"process_refund": 1,
                            "check_order": 3})],
)

result = agent(
    "Help me with my refund",
    limits={"turns": 10, "total_tokens": 8000},
)
```

Context Engineering is not optional

- Give agent just the info it needs when it needs it
- Increases reliability and reduces model confusion
- Stop spending money on tokens you don’t actively need
- Prompt caching

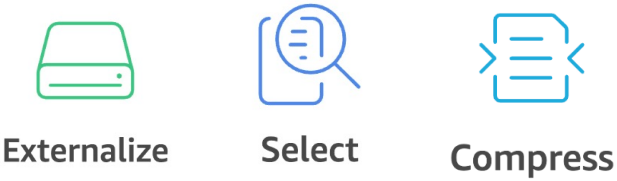
Context Engineering Strategies



Offload Large Tool Results

- Tool results can be verbose and eat up your context window
- Most of it is not relevant to every single turn of the agent loop
- By default, offload tool result to external file
- Allow agent to dynamically bring in relevant tool results as context when needed

One-line default for context engineering



- Context Offloading
- Summarization with proactive compression
- Costs dropped 55% while accuracy went from 68% to 98%

```
from strands import Agent

agent = Agent(context_manager="auto")
```

What we covered

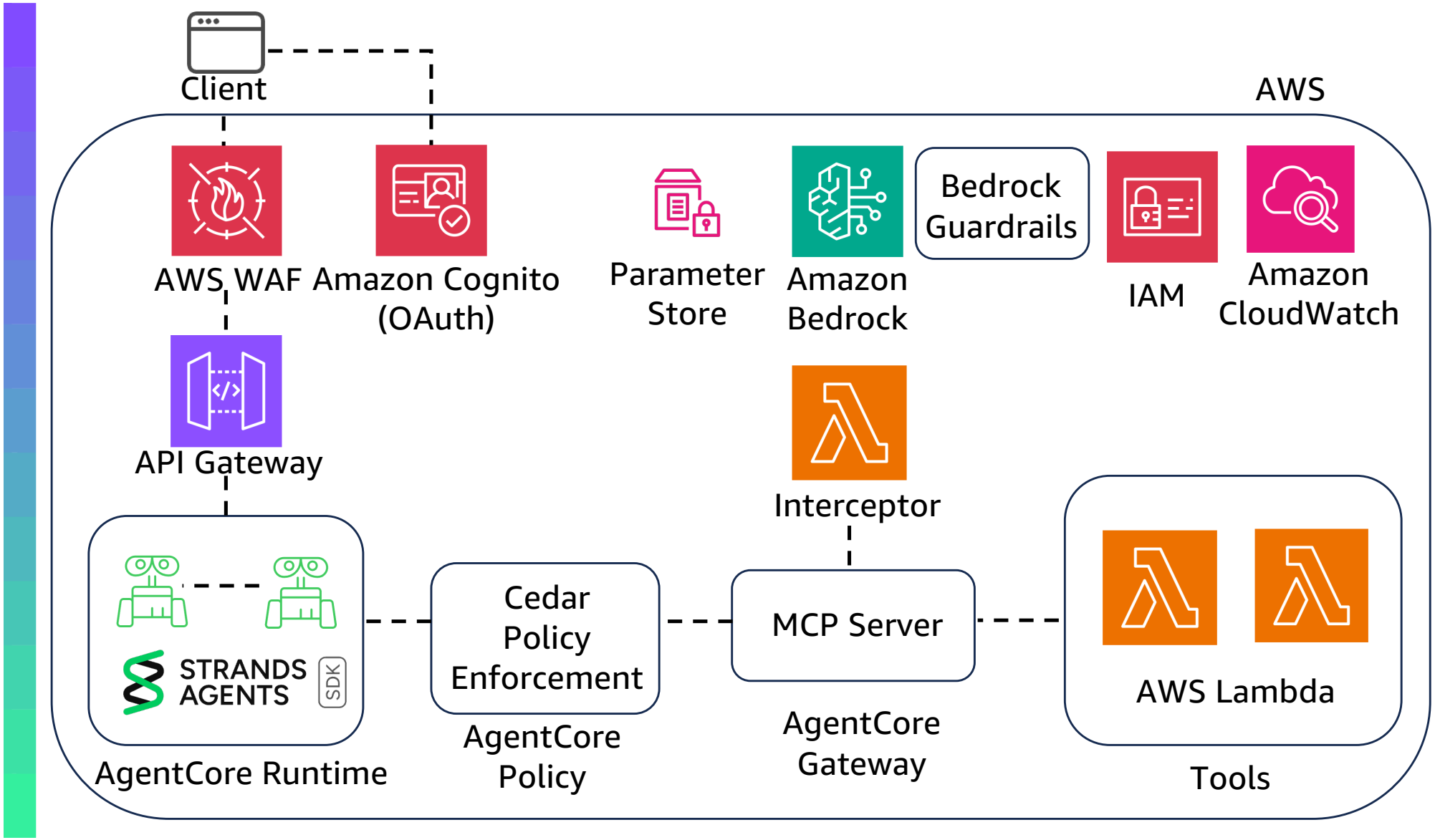
Says something wrong → Guardrails + Steering

Does something wrong → Steering + Interceptors + Cedar

Adversarial abuse → Guardrails + Steering + Cedar

Costs too much → Limits + Hooks

Applicable to all → Evaluations + Observability


Client
AWS
AWS WAF
Amazon Cognito (OAuth)
Parameter Store
Amazon Bedrock
Bedrock Guardrails
IAM
Amazon CloudWatch
API Gateway
Interceptor
STRANDS AGENTS
SDK
Cedar Policy Enforcement
MCP Server
AWS Lambda
AgentCore Runtime
AgentCore Policy
AgentCore Gateway
Tools

Go Fourth and Build Agents (Safely)

@morganwilliscloud